Testing Strategy

Version: 1.0

Purpose

This document defines the testing strategy for Trading Platform Pro.
The objective is to ensure correctness, stability, maintainability and confidence in every code change throughout the lifetime of the project.

Testing Philosophy

Testing is an integral part of software development.
Every implementation should be verifiable through automated tests wherever practical.
Testing is intended to:
• prevent regressions
• validate business behaviour
• support refactoring
• improve software quality

Test Pyramid

Trading Platform Pro follows the classical test pyramid.
```
System Tests
Integration Tests
Unit Tests
```
The majority of tests should be unit tests.

Unit Tests

Purpose:
Verify individual components in isolation.
Characteristics:
• fast
• deterministic
• independent
• no external systems
Typical targets:
• domain logic
• services
• value objects
• utilities
• infrastructure abstractions

Integration Tests

Purpose:
Verify collaboration between components.
Examples:
• repositories
• configuration loading
• dependency injection
• messaging

• persistence

System Tests

Purpose:
Validate complete workflows.
Examples:
• application startup
• runtime initialization
• trading workflow
• configuration loading
• plugin initialization

Test Organization

```
tests/
■■■ unit/
■■■ integration/
■■■ system/
■■■ performance/
```

The directory structure should mirror the production code where practical.

Test Naming

Test files:
```
test_.py
```

Examples:
```
test_cache.py
test_scheduler.py
test_runtime.py
```

Test Design

Tests should be:
• readable
• deterministic
• isolated
• maintainable
Each test should verify a single behaviour.

Mocking

Mock only external dependencies.
Avoid mocking business logic.
Examples:
• file system
• databases
• broker APIs
• network services

• system clock

Regression Tests

Every bug fix should include a regression test whenever practical.

Continuous Integration

Every push should execute the automated test suite.
Failed tests block integration.

Local Development

Recommended workflow:
```
pytest tests/unit -q
```
Run the complete test suite before merging significant changes.

Test Quality Principles

A good test is:
• simple
• stable
• fast
• independent
• easy to understand

Test Coverage

Every new feature should include appropriate test coverage.
Preferred priorities:
• business logic
• public APIs
• critical workflows
• error handling
• regression scenarios
Coverage is a quality indicator, not a goal by itself.

Performance Testing

Performance tests should verify:
• startup time
• runtime performance
• memory consumption
• scalability of critical components
Run performance tests separately from unit tests.

Paper Trading Validation

Trading-related implementations should be validated in PAPER mode before production use.
Validation should include:
• startup
• configuration

• trading workflow
• logging
• error handling
LIVE validation requires explicit approval.

Test Review

Before merging verify:
• tests remain deterministic
• no flaky tests
• meaningful assertions
• no duplicated tests
• clear naming
• maintainable structure

Related Documents

• Development_Guidelines.md
• Coding_Standards.md
• Git_Workflow.md
• AGENTS.md